



[BIT703 Web Technologies](#) / [Contents](#) / Topic 11: Server-side web application admin panel development /
Authorisation and authentication

Authorisation and authentication in an admin panel

10m

According to the Open Web Application Security Project (OWASP, 2021), broken access control was the number one web application security risk. In 2025, this is still an issue, as discussed in One weak password brought down a 158-year-old company (Knight, 2025).

Broken access control includes many different aspects, but the key issue can be summarised as users in some way accessing resources that they should not have access to. In this case, broken access control would be a user who does not have the correct administration permissions or access level gaining access to the admin panel.

Two separate but complementary processes manage access to web resources: authentication and authorisation.

- **Authentication:** Identifying who a user is. This would mean the user having an administration account in the system that is secured in some way, typically with a username and password, and possibly other mechanisms such as biometrics (for example, face or voice recognition).
- **Authorisation:** Once a user has been identified, they will be authorised to access the admin panel. This is usually done by role. Each user will be logged in with a specific role, defined by their account, and that role will be authorised to access a specified set of resources.

[Skip to main content](#)

To demonstrate how these processes can be managed using PHP, we will continue to follow the admin panel we've created for Kākahu's online store and include both authentication and authorisation to it by using sessions as the mechanism for tracking the login state of the user.

Sessions are a way for a web application to maintain a link between pages to overcome the stateless nature of HTTP. It can also be used to track authentication. PHP has extensive support for session management.

If you want more technical details about the relationships between authentication, authorisation and sessions, then you may find this [Session Management Cheat Sheet \(OWASP Cheat Sheet Series, n.d.\)](#) useful.

Let's begin by looking at password encryption. Different types of encryptions require different sizes of storage in the database. In a database, the recommended length for an encrypted/hashed password is 60 characters with VARCHAR(255) providing ample storage (Codecademy, n.d.). Because the password field is currently defined with a length of 150, the **users table** could be updated to a length of 255 characters for passwords.

This dropping and replacing of tables is, of course, only appropriate during development and testing and not the sort of approach that would be used with a deployed system.

Password encryption using PHP

In order to encrypt a password using PHP, you will need a PHP page that creates example data records that include the encrypted passwords.

In the following examples, we will add two users to the database using PHP code rather than using a SQL query. The critical element of the PHP code is the **password_hash** function, which is used to encrypt data (in this case, it is encrypting the password):

```
$ hashed_password = password_hash($password, PASSWORD_DEFAULT);
```

[Skip to main content](#)

The first argument to this `password_hash` function is the password to be encrypted, and the second is the encryption algorithm. This example uses the PHP default which aligns with the MySQL encryption and hashing.

In the example `login.php`, the PHP code is used to add an admin user (an administrator with complete access to the admin panel) and an ordinary user of the online store (a customer with no access to the admin panel but access to buy products from Kākahu's online store) to the user table in the database.

This can be put into a PHP page and run on the server (again, this is not the sort of code you would use for deployment; only development and testing).

An admin user should have access to all the admin pages in the web application, including the admin panel, whereas a customer who does not have an admin role should only have access to the customer pages.

This decision is not made at the authentication level but at the authorisation level. The admin user and customer user will be sufficient for testing authentication, which we will look at now.



Thoughts on this page?

